Warm-Up: Basic Operations

1. **Lists:**
   - Create a list of integers from 1 to 10. Write a script to:
     - Print the first and last elements.
     - Reverse the list.
     - Find the sum of all elements

In [ ]:
```
def list_create(num):
    """Function to create list of integers from 1 to num"""
    return [integer for integer in range(num + 1)]


def get_first_and_last(list1):
    """Function to print the first and last element"""
    print(f"First element: {list1[0]}\nLast element:{list1[-1]}")


def list_reverse(list1):
    """Function to reverse the list"""
    reversed_list = [list1[i] for i in range(-1,- len(list1),-1)]
    print(reversed_list)


def list_sum(list1):
    """Function to get the sum of all values in the list of integers"""
    total = 0
    for num in list1:
        total += num
    print(f"Sum of all elements is: {total}")


list1 = list_create(10)
get_first_and_last(list1)
list_reverse(list1)
list_sum(list1)
```

```
First element: 0
Last element:10
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
Sum of all elements is: 55
```

2. **Dictionaries:**
   - Create a dictionary where keys are the first 5 natural numbers and values are their squares. Write a script to:
     - Print all keys and their respective values.
     - Add a new key-value pair: `6: 36`.
     - Check if a specific key (e.g., 4) exists in the dictionary.

In [ ]:
```
def dict_add(squares_dict,i):
    """Add a new key_value pair"""
    squares_dict[i] = i ** 2

def dict_square_create():
    """Create a dictionary where keys are the first 5 natural numbers and values are their squ
    squares_dict = {}
    for i in range(1,6):
        dict_add(squares_dict ,i)
```

```
    print(squares_dict.items())
    return squares_dict


def check_squares_list(squares_dict , num):
    """Check if a specific key exists in the dictionary"""
    if squares_dict.get(num, 'None') != 'None':
        print(f"The key {num} Exists!")
    else:
        print(f"The key {num} Doesn't Exist")



squares_dict = dict_square_create()
dict_add(squares_dict,6)
print(f"After adding : {squares_dict}")
squares_dict.get(7)
check_squares_list(squares_dict , 4)
```

```
dict_items([(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)])
After adding : {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36}
The key 4 Exists!
```

Problem-Solving: Real-World Scenarios

1. **Student Grades:**
   - Create a dictionary to store student names as keys and a list of their grades as values. Write a script to:
     - Calculate the average grade for each student.
     - Find the student with the highest average

In [5]:
```
def student_avg(values):
        """Get average mark of 1 student"""
        total = 0
        for i in values:
            total += i
        avg = total / len(values)
        return avg


def average_grade(grade_dict):
    """Calculate the average grade for each student"""
    student_avg_dict = {}
    for name ,values in grade_dict.items():
        avg = student_avg(values)
        student_avg_dict[name] = avg
    return student_avg_dict


def highest_avg(grade_dict):
    """Get the maximum average in marks"""
    student_avg_dict = average_grade(grade_dict)
    max_avg = max(student_avg_dict.values())
    max_student = max(student_avg_dict, key=student_avg_dict.get)
    print(f"\nThe max average mark is {max_avg} belonging to student {max_student}")



grade_dict = {'yoyo' : [49,50,50], 'roro' : [48,50,49] ,'koko' : [10,30,15] , 'soso' : [30,48
average_dict = average_grade(grade_dict)
for name, avg in average_dict.items():
    print(f"The average mark of student {name} is {avg}")
highest_avg(grade_dict)
```

```
The average mark of student yoyo is 49.666666666666664
The average mark of student roro is 49.0
The average mark of student koko is 18.333333333333332
The average mark of student soso is 33.333333333333336

The max average mark is 49.666666666666664 belonging to student yoyo
```

2. **Shopping Cart:**
   - Implement a shopping cart using a dictionary where keys are product names and values are their prices. Write a script to:
     - Add items to the cart.
     - Calculate the total price of items in the cart

In [61]:
```
def cart_add(cart_dict, product, price):
    cart_dict[product] = price


def calc_total_price(cart_dict):
    total = 0
    for price in cart_dict.values():
        total += int(price)
    print(f"The total price of items in cart is: {total}")




cart_dict = {'cucumber' : '8' , 'rice' : '29' , 'oil' : '56' , 'salt' : '2' , 'pepper' : '80'
calc_total_price(cart_dict)
cart_add(cart_dict, 'tomato', '12')
print (f"After adding: {cart_dict}")
calc_total_price(cart_dict)
```

```
The total price of items in cart is: 175
After adding: {'cucumber': '8', 'rice': '29', 'oil': '56', 'salt': '2', 'pepper': '80', 'tomat
o': '12'}
The total price of items in cart is: 187
```

3. **Employee Database:**
   - Create a list of dictionaries where each dictionary represents an employee with attributes: `name` , `age` , and `department` . Write a script to:
     - Filter employees by a specific department.
     - Find the average age of employees.

In [ ]:
```
def filter_by_department(employees , department):
    """"Check for people in the same departement"""
    list1 = []
    for employee in employees:
            if employee['department'] == department:
                list1.append(employee['name'])
    print(f"Employees in department{department} are {list1}")


def avg_age(employees):
    """Get Average age of all employees"""
    list1 = []
    for employee in employees:
        list1.append(int(employee['age']))
    avg_age = sum(list1)/len(list1)
    print(f"Average age is :{avg_age}")

employee1 = {'name' : 'Ahmed' , 'age' : '30' , 'department' : 'Finance'}
```

```
employee2 = {'name' : 'Mohammed' , 'age' : '28' , 'department' : 'IT'}
employee3 = {'name' : 'Sara' , 'age' : '40' , 'department' : 'IT'}
employee4 = {'name' : 'Lamiaa' , 'age' : '45' , 'department' : 'Marketing'}
employee5 = {'name' : 'Farah' , 'age' : '25' , 'department' : 'Finance'}
employees = [employee1 , employee2 , employee3 , employee4 ,employee5]


filter_by_department(employees , 'IT')
avg_age(employees)
```

```
Employees in departmentIT are ['Mohammed', 'Sara']
Average age is :33.6
```

Challenge Section

1. **Word Frequency Counter:**
   - Given a string, count the frequency of each word using a dictionary

In [1]:
```
def count_freq(string1):
    """Count the frequency of each word using dictionary"""
    words = string1.split() #list of the words in the string
    print(words)
    freq = {}
    for word in words:
        if word in freq:
            freq[word] += 1
        else:
            freq[word] = 1
    print(freq)


string1 = "Hello , say Hello or hi to Salma or Sama , Salma will respose."
count_freq(string1)
```

```
['Hello', ',', 'say', 'Hello', 'or', 'hi', 'to', 'Salma', 'or', 'Sama', ',', 'Salma', 'will',
'respose.']
{'Hello': 2, ',': 2, 'say': 1, 'or': 2, 'hi': 1, 'to': 1, 'Salma': 2, 'Sama': 1, 'will': 1, 'r
espose.': 1}
```

3. **Nested Data Manipulation:**
   - Create a nested dictionary to represent a library where keys are genres and values are lists of books. Write a script to:
     - Add a new book to a genre.
     - List all books of a specific genre

In [99]:
```
library = {'Comedy' : ['Alice' , 'Bob' , 'Alien']
            , 'horror' : ['ghost', 'devils']
            , 'Drama' : ['bebo' ,'jojo', 'lili', 'oppo']}


def add_new_book(library , genre , book):
    """To add new booh=k to the genre"""
    library[genre].append(book)
    print(library)

def list_books(library , genre):
    """To list all books of a specific genre"""
    for book in library[genre]:
        print(book)

##Or:
def list_books2(library , genre):
```

```
    """To list all books of a specific genre"""
    print(library['Comedy'][:])

add_new_book(library , 'Comedy' , 'henedi')
list_books(library , 'Comedy')
list_books2(library , 'Drama')
```

```
{'Comedy': ['Alice', 'Bob', 'Alien', 'henedi'], 'horror': ['ghost', 'devils'], 'Drama': ['beb
o', 'jojo', 'lili', 'oppo']}
Alice
Bob
Alien
henedi
['Alice', 'Bob', 'Alien', 'henedi']
```

Additional

2. **Inventory Management:**
   - Create an inventory system for a store using a dictionary where keys are item names and values are dictionaries containing `price` and `quantity`. Write a script to:
     - Update stock after a sale.
     - Restock items

In [100…

```
def update_inventory(inventory , key ,sold):
    """Update stock after sale"""
    inventory[key][-1] -= sold

def restock_inventory(inventory , key ,quantity):
    """Restock the items"""
    inventory[key][-1] += quantity

inventory = {'mobile' : [1000 , 100] , 'speaker' : [5000 , 8] , 'tablet' : [1500 , 40]}
update_inventory(inventory , 'mobile' , 3)
print(inventory)
restock_inventory(inventory , 'tablet' , 10)
print(inventory)
```

```
{'mobile': [1000, 97], 'speaker': [5000, 8], 'tablet': [1500, 40]}
{'mobile': [1000, 97], 'speaker': [5000, 8], 'tablet': [1500, 50]}
```

4. **Anagram Checker:**
   - Write a function to check if two given strings are anagrams of each other. Two strings are anagrams if they use the same characters in the same frequency, regardless of order.
   - Example:
     Input: "listen", "silent"
     Output: True

In [6]:

```
def check_anagram(str1 ,str2):
    """Check if 2 strings are anagrams"""
    if sorted(str1) == sorted(str2):
        return True
    else:
        return False


str1 = "listen"
str2 = "silent"
check_anagram(str1 ,str2)
```

Out[6]: True

5. **Two-Sum Problem:**
   - Write a function that takes a list of integers and a target number and returns the indices of two numbers in the list that add up to the target.
   - Example:
     Input: nums = [2, 7, 11, 15], target = 9
     Output: [0, 1]

In [ ]:
```
def target_add(list1 , target):
    """Gets the indices of two numbers in list if added equals the target number"""
    output = []
    for i in range(len(list1)): # 0  1  2  3
        for j in range(i + 1 , len(list1)): # 1 2 3
            if list1[i] + list1[j] == target:
                output.append(i)
                output.append(j)
    print(output)



list1 = [2, 7, 11, 15 ]
target_add(list1 , 9)
```

```
[0, 1]
```